

Ingeniería de Software I

Avance de proyecto # 3 – Arquitectura inicial

Fecha de asignación: viernes 04 de septiembre de 2026.

Docente: Juan Pablo Bustamante Moreno.

Escuela Tecnológica Instituto Técnico Central (ETITC).

En los avances anteriores, cada equipo definió la propuesta de producto, requisitos funcionales y no funcionales, restricciones, reglas de negocio, historias de usuario, Product Backlog, casos de uso y mapa de stakeholders.

En este avance deberán proponer una primera organización técnica del sistema. El objetivo es cerrar la fase de planeación con una arquitectura inicial, un modelo de datos y una selección tecnológica que permita iniciar el desarrollo por Sprints en las semanas posteriores. No se que la arquitectura sea definitiva, sino una propuesta clara, viable y justificable que podrá evolucionar durante el desarrollo.

Instrucciones generales

- Continúen trabajando en el mismo documento compartido del proyecto.
- Agreguen las secciones de este avance después del Avance de Proyecto #2.
- Utilicen diagramas sencillos y legibles. Pueden emplear Draw.io, Lucidchart, PlantUML, Excalidraw, Figma, Visual Paradigm u otra herramienta equivalente.
- No seleccionen tecnologías, patrones o estilos arquitectónicos únicamente por tendencia. Para la mayoría de los proyectos del curso, una arquitectura en capas, MVC o cliente-servidor será suficiente si se encuentra bien justificado.
- Las decisiones de este avance pueden cambiar durante los Sprints, pero cualquier cambio importante deberá ser documentado posteriormente.

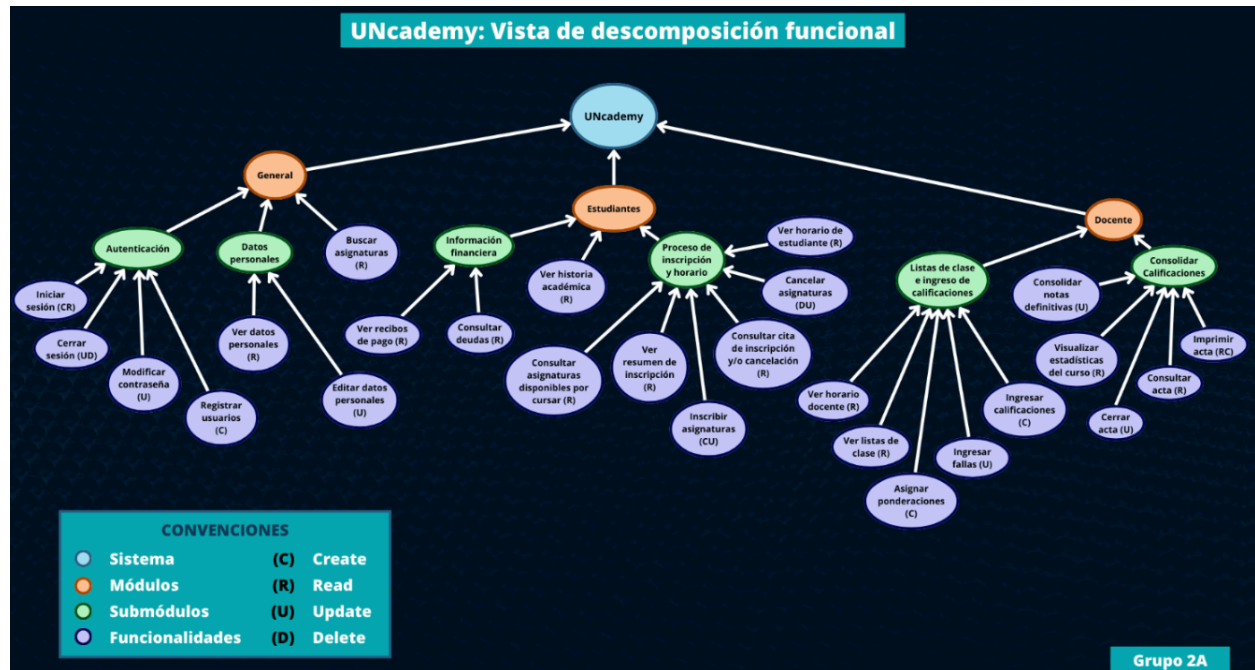
1. Vista de descomposición funcional

A partir de las funcionalidades que definieron en su backlog, construyan una representación gráfica de estas y su agrupación en módulos y submódulos. El propósito de esta vista es transformar la lista de historias de usuario en una imagen general del producto.

La vista debe incluir:

- Nombre del sistema.
- Entre tres y seis módulos, áreas funcionales o capacidades principales.
- Funcionalidades relevantes agrupadas en cada módulo (u organizadas por submódulos, si aplica). Esta relación se puede derivar de las épicas definidas en el Avance #2.

Ejemplo conceptual:



2. Vista del modelo de datos

Construyan una primera versión del modelo de datos del proyecto. Pueden utilizar:

- Diagrama Entidad-Relación.
- Modelo lógico relacional.
- Modelo de documentos o colecciones, si justifican una base de datos NoSQL.

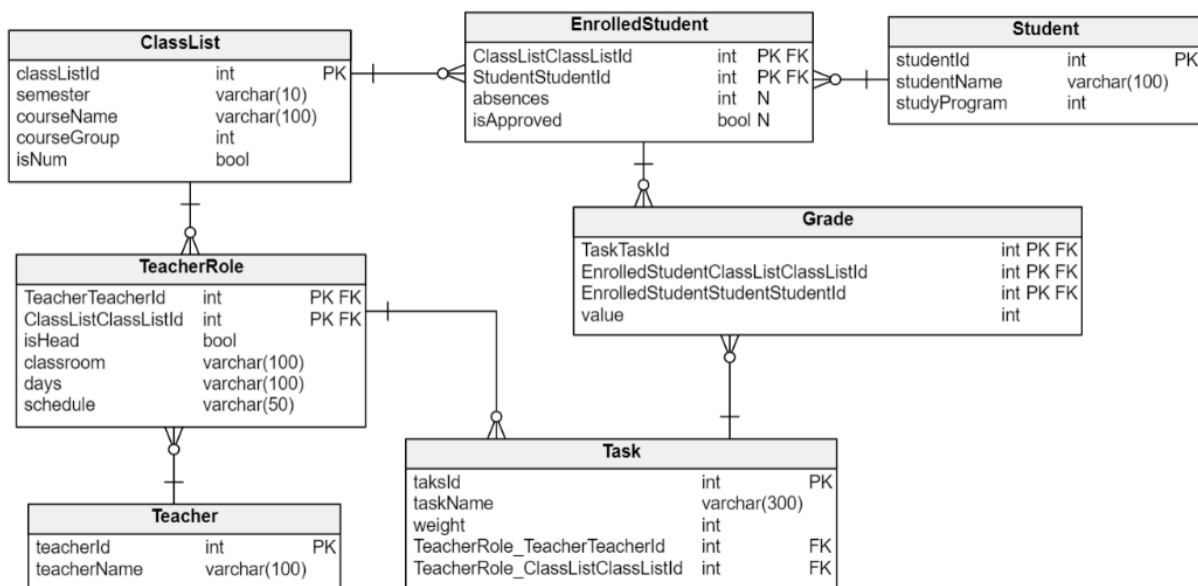
Si el proyecto utilizará una base de datos relacional, el modelo debe incluir:

- Entidades.
- Atributos principales.
- Llaves primarias.
- Relaciones y cardinalidades.
- Llaves foráneas cuando apliquen.
- Restricciones de integridad importantes.

Recomendaciones:

- Incluyan solamente las entidades necesarias para el alcance definido en su planeación del Product Backlog, dando prioridad a los datos requeridos por las funcionalidades principales del producto.
- Revisen que el modelo de datos se relacione con los requisitos funcionales, las historias priorizadas y las reglas de negocio.
- Si una regla de negocio afecta la integridad de los datos, pueden indicarla como una nota junto al modelo.

Ejemplo conceptual:



3. ADR-001: Arquitectura y selección tecnológica inicial

El equipo debe elaborar su primer *Architecture Decision Record* (ADR). Este artefacto registrará la decisión inicial sobre cómo se organizará técnicamente el sistema y qué tecnologías utilizará el equipo. Debe ser claro, concreto y argumentado.

La pregunta central de este ADR es:

¿Qué arquitectura y conjunto tecnológico inicial permite desarrollar esta solución de manera viable durante el semestre?

Pueden emplear el siguiente esquema, para redactarlo a manera de texto:

Campo	Contenido requerido
Estado	Propuesto o aceptado
Fecha	Fecha de elaboración
Contexto	Tipo de solución, requisitos relevantes, restricciones y atributos de calidad que influyen en la decisión
Decisión	Enfoque arquitectónico y tecnologías seleccionadas
Alternativas consideradas	Mínimo dos alternativas
Justificación	Razones de viabilidad, capacidad del equipo, alcance, requisitos y calidad
Consecuencias positivas	Beneficios esperados
Consecuencias, riesgos o limitaciones	Riesgos técnicos, dependencias, trade-offs, incertidumbres o aspectos que deben validarse durante Sprint 1

Dentro de la sección “Decisión”, incluyan una tabla que dé cuenta del enfoque y las herramientas elegidas por el grupo. Por ejemplo:

Área	Decisión preliminar
Tipo de solución	Web, móvil, escritorio, híbrida u otra
Enfoque arquitectónico	Capas, MVC, cliente-servidor, monolito modular, o algún otro
Lenguaje	
Framework o librería principal	
Interfaz o cliente	
Backend o API	

Persistencia	SQL, NoSQL, ORM, DAO/Repository u otra
Motor de base de datos	PostgreSQL, MySQL, MongoDB u otro
Contenerización	Docker, Docker Compose, Kubernetes u otro
Gestión de trabajo	Trello, GitHub Projects, Jira, Notion u otra
Herramientas de modelado/documentación	
Pruebas previstas	Pruebas unitarias, de integración, API u otras; manuales, automatizadas o híbridas
Despliegue previsto	Local, contenedor, servidor, nube u otro

Dentro de la sección “Consecuencias, riesgos o limitaciones” pueden hacer una matriz como esta, con al menos **cinco riesgos**:

ID	Riesgo técnico o arquitectónico	Impacto	Probabilidad	Estrategia inicial
RTA-01		Alto / Medio / Bajo	Alto / Medio / Bajo	Mitigar / investigar / simplificar / otro
RTA-02				
...				

No incluyan riesgos genéricos como “puede faltar tiempo” sin explicar qué aspecto técnico puede causar el retraso y qué acción tomarán. Algunos ejemplos válidos:

Riesgo	Estrategia inicial posible
El equipo no tiene experiencia con el framework seleccionado	Realizar un spike técnico inicial y limitar el alcance de Sprint 1

La integración con una API externa puede no estar disponible	Diseñar una interfaz de integración y crear datos simulados como plan alternativo
Una funcionalidad depende de un servicio externo por el que se debe pagar	Construir un flujo simulado para la primera versión; consultar convenios institucionales

Finalmente, como complemento del ADR, incluyan un diagrama sencillo que represente los elementos técnicos principales y su relación. Debe mostrar, según aplique

- Usuario, actor o cliente.
- Interfaz web, móvil, de escritorio, consola o API.
- Backend, aplicación o servicio principal.
- Módulos funcionales relevantes.
- Base de datos.
- Sistemas externos, APIs o servicios de terceros.
- Contenedores o entornos de ejecución, si ya fueron definidos.

No es necesario que sea un diagrama UML formal, sino una representación gráfica de su arquitectura inicial (evidenciando los elementos que tendrá el sistema y las relaciones entre ellos). También pueden agregar un párrafo explicando su diagrama si lo consideran pertinente.

Entrega

Actualicen el documento compartido de OneDrive creado para los avances de proyecto #1 y 2. No deben crear un archivo nuevo: el documento del proyecto va a crecer de manera progresiva durante todo el semestre.

Este avance no se califica de forma independiente. Será revisado y retroalimentado durante el Sprint 0, y hará parte de la evidencia evaluada en la sustentación del primer corte, que se realizará el viernes 11 de septiembre en el horario de clase (tras la sustentación, tendrán plazo hasta el lunes 14 de septiembre para completar o corregir los entregables que hagan falta).